```
    val askFlow: Flow[Int, Int, NotUsed] =        ❷
      ActorFlow.ask(ref)((elem: Int, replyTo: ActorRef[Int]) =>
        Cap.Increment(elem, replyTo))

    val result: Future[Done] = Source(1 to 10)
      .via(askFlow)                               ❸
      .map(println)
      .run()

    Await.result(result, 1.seconds)
    }
  }
  ...
}
```

❶ Actor to ask

❷ Flow that represents asking

❸ Flow usage

After you spawn the `Cap` actor, you can use its reference to `ask` with the `ActorFlow`.

TIP There is a shortened version of `ask` (syntactic sugar) that you should use. We used the previous notation for didactic reasons to show you the element and the proxy actor. But you can forget the tuple and pass the asking message to the constructor: `ActorFlow.ask(ref)(Cap.Increment)`. The compiler can derive the rest based on the message constructor.

It is worth noting that `ActorFlow.ask` has a parallelism of 2. If you send a message with a relatively high payload, you should consider greater parallelism, like this:

```
ActorFlow.ask(10)(ref)(Cap.Increment)
```